

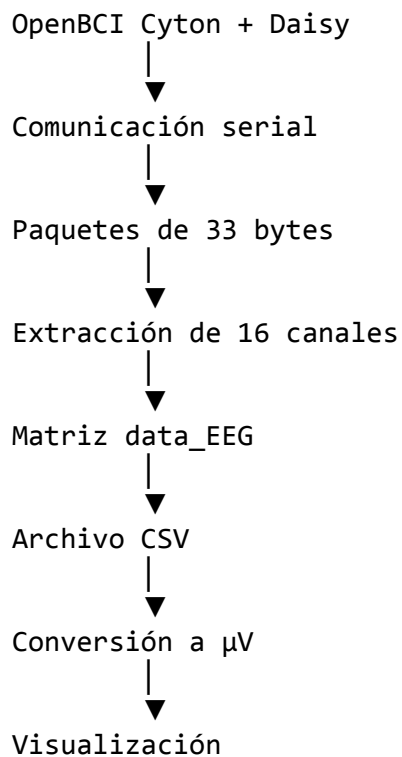
Adquisición y almacenamiento de datos EEG

1. Introducción

Este documento describe el proceso mediante el cual el programa adquiere, organiza y almacena las señales EEG provenientes del **OpenBCI Cyton + Daisy**.

La adquisición se realiza mediante comunicación serial desde una Raspberry Pi utilizando Python.

El proceso general es:



2. Configuración de la comunicación

La comunicación serial se establece mediante PySerial:

```
ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
```

Los parámetros utilizados son:

Parámetro	Valor
Puerto	/dev/ttyUSB0

Parámetro	Valor
Baud rate	115200
Timeout	1 segundo

Después de abrir el puerto, el programa espera dos segundos:

```
time.sleep(2)
```

Esto permite que la comunicación con el dispositivo se establezca antes de enviar comandos.

3. Inicialización del OpenBCI

El programa inicia la comunicación con:

```
print("Iniciando Cyton...")
```

Posteriormente envía el comando:

```
ser.write(b'v')
```

que se utiliza para reinicializar el dispositivo.

Después de esperar un segundo:

```
time.sleep(1)
```

se envía:

```
ser.write(b'd')
```

para utilizar la configuración predeterminada de los canales.

Finalmente se establece en el programa:

```
fs = 250
```

Este valor se utiliza posteriormente para construir el eje temporal de la gráfica.

Nota: `fs` en este código funciona como referencia para interpretar temporalmente las muestras y no constituye por sí mismo una modificación de la frecuencia física de adquisición del hardware.

4. Inicio del streaming

Antes de comenzar la adquisición se limpia el búfer de entrada:

```
ser.reset_input_buffer()
```

Esto evita procesar datos que pudieran haber quedado almacenados en el puerto serial antes del inicio de la adquisición.

Después se envía:

```
ser.write(b'b')
```

para comenzar el streaming.

El programa muestra:

```
print("Adquiriendo datos...")
```

indicando que comenzará la recepción de muestras.

5. Número de muestras

El número de muestras que serán adquiridas se establece mediante:

```
N_points = 1000
```

Por lo tanto, el programa continúa adquiriendo datos hasta obtener:

1000 muestras

para cada uno de los 16 canales.

La cantidad total de valores EEG almacenados será:

16 canales × 1000 muestras

es decir:

16000 valores

6. Creación de la matriz de datos

Para almacenar las muestras se crea:

```
data_EEG = np.zeros((16, N_points))
```

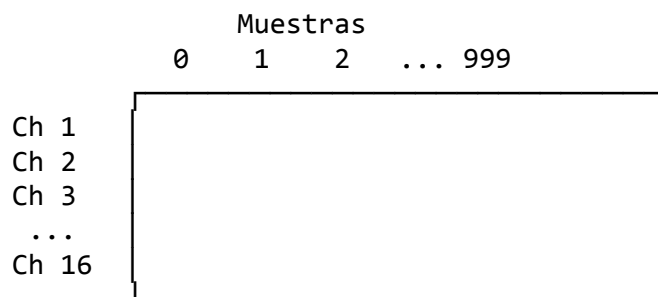
Como:

```
N_points = 1000
```

la matriz tiene dimensiones:

(16, 1000)

La primera dimensión corresponde a los canales y la segunda a las muestras.



Por lo tanto:

```
data_EEG[ch, i]
```

significa:

```
canal = ch  
muestra = i
```

7. Índice de muestra

El programa utiliza:

```
i = 0
```

para comenzar con la primera muestra.

Posteriormente se utiliza:

```
while i < N_points:
```

El ciclo continúa mientras:

```
i < 1000
```

Cada vez que el programa obtiene correctamente los datos correspondientes a los 16 canales, incrementa:

```
i += 1
```

Por lo tanto:

```
i = 0  
↓  
i = 1  
↓
```

```
i = 2
↓
...
↓
i = 999
↓
i = 1000
↓
fin
```

Cuando i llega a 1000, la condición:

$i < N_points$

deja de cumplirse y termina la adquisición.

8. Lectura de los paquetes

Dentro del ciclo se leen dos paquetes:

```
packet1 = ser.read(33)
packet2 = ser.read(33)
```

Cada llamada solicita:

33 bytes

Por lo tanto, en cada iteración se reciben:

$33 + 33 = 66$ bytes

Estos paquetes contienen los datos necesarios para obtener los 16 canales utilizados por el programa.

La distribución lógica es:

```
packet1
├── Canal 1
├── Canal 2
├── ...
└── Canal 8
```

```
packet2
├── Canal 9
├── Canal 10
├── ...
└── Canal 16
```

9. Validación de los paquetes

Antes de procesar los datos, el programa verifica que ambos paquetes tengan exactamente 33 bytes:

```
if len(packet1) == 33 and len(packet2) == 33:
```

Esta condición evita procesar paquetes incompletos.

Por ejemplo, si se recibe:

packet1 → 33 bytes

packet2 → 27 bytes

la condición no se cumple y los datos no se almacenan.

10. Validación del encabezado

Después de verificar el tamaño se comprueba el primer byte de cada paquete:

```
if packet1[0] == 0xA0 and packet2[0] == 0xA0:
```

El valor:

0xA0

corresponde al encabezado esperado de un paquete de datos Cyton.

La condición garantiza que ambos paquetes comiencen con el encabezado esperado antes de extraer los canales.

11. Extracción de los canales 1–8

Los primeros ocho canales se obtienen de packet1:

```
for ch in range(8):
    idx = 2 + ch*3

    data_EEG[ch, i] = read_24bit_signed(
        packet1[idx],
        packet1[idx+1],
        packet1[idx+2]
    )
```

El ciclo:

```
range(8)
```

produce:

```
0, 1, 2, 3, 4, 5, 6, 7
```

Cada iteración corresponde a un canal.

El cálculo:

```
idx = 2 + ch*3
```

permite localizar los tres bytes correspondientes al canal.

La información se convierte mediante:

```
read_24bit_signed()
```

y finalmente se almacena en:

```
data_EEG[ch, i]
```

12. Extracción de los canales 9–16

Los ocho canales restantes se obtienen de packet2:

```
for ch in range(8):  
    idx = 2 + ch*3  
  
    data_EEG[ch+8, i] = read_24bit_signed(  
        packet2[idx],  
        packet2[idx+1],  
        packet2[idx+2]  
    )
```

El desplazamiento:

```
ch + 8
```

permite utilizar las posiciones correspondientes a los canales 9–16.

La relación es:

ch	ch + 8	Canal
0	8	9
1	9	10
2	10	11

ch	ch + 8	Canal
3	11	12
4	12	13
5	13	14
6	14	15
7	15	16

13. Construcción de una muestra de 16 canales

Una iteración completa del ciclo `while` construye una posición temporal `i`.

Por ejemplo, cuando:

```
i = 0
```

se almacenan:

```
data_EEG[0,0] → Canal 1
data_EEG[1,0] → Canal 2
data_EEG[2,0] → Canal 3
...
data_EEG[15,0] → Canal 16
```

Cuando:

```
i = 1
```

se obtiene:

```
data_EEG[0,1] → Canal 1
data_EEG[1,1] → Canal 2
...
data_EEG[15,1] → Canal 16
```

Y así sucesivamente hasta llegar a:

```
i = 999
```

14. Representación de la matriz final

Una vez terminada la adquisición:

```
data_EEG.shape
```

produce:

(16, 1000)

La matriz puede visualizarse conceptualmente como:



Cada fila contiene toda la adquisición de un canal.

Cada columna representa una posición de muestra dentro de la adquisición.

15. Finalización de la adquisición

Cuando se obtienen las 1000 muestras, el ciclo termina.

El programa envía:

```
ser.write(b's')
```

para detener el streaming.

Posteriormente se cierra el puerto serial:

```
ser.close()
```

Esto libera el dispositivo y finaliza la comunicación serial.

16. Verificación de los datos

Después de cerrar la comunicación se muestra:

```
print("Datos obtenidos:", data_EEG.shape)
```

Si la adquisición fue completada correctamente, se espera:

Datos obtenidos: (16, 1000)

Esto confirma que el programa tiene una matriz con:

16 canales
1000 muestras por canal

17. Preparación para guardar los datos

Para guardar los datos utilizando Pandas se realiza una transposición:

`data_EEG.T`

Originalmente:

16×1000

Después de la transposición:

1000×16

Esto es conveniente para crear el archivo CSV porque cada fila representará una muestra y cada columna representará un canal.

La estructura pasa de:

Canales $\times$ Muestras

a:

Muestras $\times$ Canales

18. Creación del DataFrame

El DataFrame se crea mediante:

```
df = pd.DataFrame(  
    data_EEG.T,  
    columns=[f"Canal_{i+1}" for i in range(16)]  
)
```

Las columnas generadas son:

Canal_1
Canal_2
Canal_3
Canal_4
Canal_5
Canal_6
Canal_7
Canal_8

```
Canal_9  
Canal_10  
Canal_11  
Canal_12  
Canal_13  
Canal_14  
Canal_15  
Canal_16
```

19. Almacenamiento en CSV

Finalmente, el DataFrame se guarda mediante:

```
df.to_csv("EEG_16_canales.csv", index=False)
```

El archivo generado es:

```
EEG_16_canales.csv
```

El parámetro:

```
index=False
```

evita que Pandas agregue una columna adicional con el índice de cada fila.

El resultado tiene conceptualmente la siguiente estructura:

```
Canal_1,Canal_2,Canal_3,...,Canal_16  
valor,valor,valor,...,valor  
valor,valor,valor,...,valor  
valor,valor,valor,...,valor  
...
```

Con 1000 muestras, el archivo contiene 1000 filas de datos y 16 columnas correspondientes a los canales.

20. Conversión a microvoltios

Después de guardar los valores originales, el programa realiza una conversión opcional:

```
scale = 4.5 / (24 * (2**23 - 1)) * 1e6  
data_EEG_uV = data_EEG * scale
```

El resultado se almacena en:

`data_EEG_uV`

Esta matriz conserva exactamente las mismas dimensiones:

`(16, 1000)`

La diferencia es que sus valores se expresan en una escala de microvoltios según el factor utilizado por el programa.

Esta conversión se utiliza para la visualización de las señales.

21. Construcción del eje temporal

Para representar las muestras en función del tiempo se utiliza:

```
t = np.arange(N_points) / fs
```

El vector `t` contiene las posiciones temporales asociadas a las muestras.

Con:

```
N_points = 1000  
fs = 250
```

el programa construye un vector temporal de 1000 posiciones.

Por ejemplo:

```
Muestra 0 → 0 / fs  
Muestra 1 → 1 / fs  
Muestra 2 → 2 / fs  
...  
Muestra 999 → 999 / fs
```

El valor de `fs` se utiliza aquí como referencia temporal para la gráfica.

22. Visualización de los 16 canales

El programa establece un desplazamiento vertical:

```
offset = 100
```

Después recorre los 16 canales:

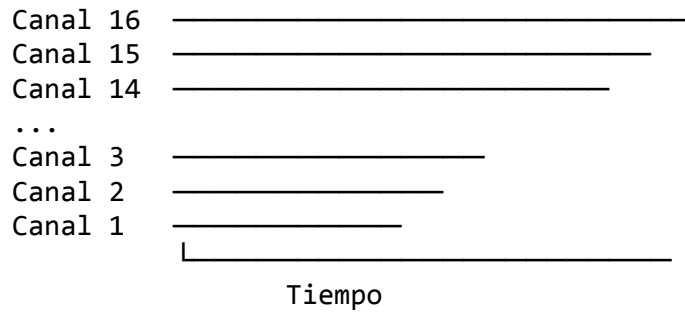
```
for ch in range(16):  
    plt.plot(t, data_EEG_uV[ch] + ch*offset)
```

El término:

`ch * offset`

se utiliza para separar visualmente las señales.

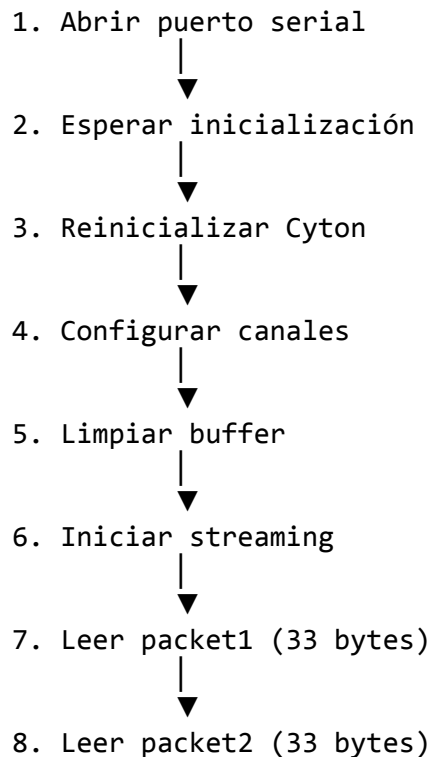
Conceptualmente:

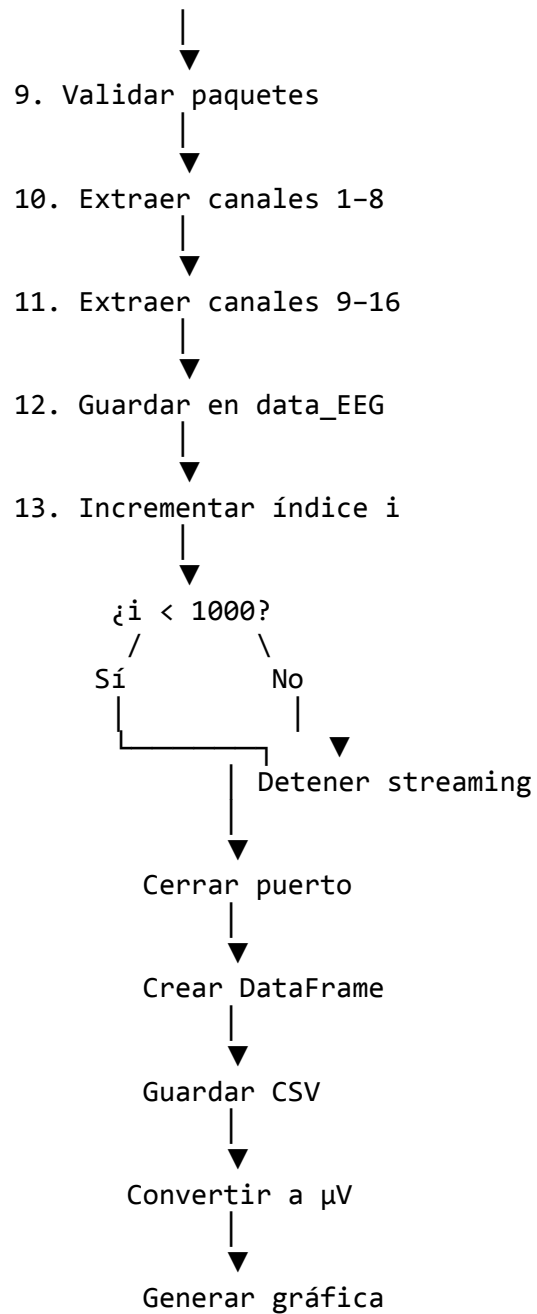


El desplazamiento no modifica los datos originales almacenados en el CSV; únicamente modifica su posición vertical en la gráfica.

23. Flujo completo de adquisición

El proceso completo implementado por el programa puede resumirse de la siguiente manera:





24. Resumen

El sistema realiza una adquisición de:

16 canales
x
1000 muestras

produciendo:

16000 valores

La información pasa por las siguientes etapas:

Datos digitales del OpenBCI



Paquetes seriales de 33 bytes



Extracción de 8 canales por paquete



Combinación de dos paquetes



16 canales



Matriz NumPy (16 × 1000)



DataFrame Pandas (1000 × 16)



EEG_16_canales.csv



Conversión opcional a μV



Visualización

Este flujo constituye el proceso principal de adquisición implementado en el proyecto.